

Trabalho Final — Guia dos 13 Temas

5 trilhas de skill · 60 pts · escolha pelo que seu grupo curte

Prof. Jackson Smith Moisés Matias

PUC Minas IEC



Trilha Automação & Arquitetura

#	Tema	Em bom português	Stack
1	Automação Mobile E2E ★★★★★	Testar o app inteiro como usuário real testaria	Maestro (cloud farm é diferencial, não obrigação)
2	Arquitetura de Suíte ★★	Organizar os testes que já escrevem, sem virar bagunça	Robot Pattern sobre RNTL
5	Visual Regression Testing ★★	Pega quando a tela quebrou visualmente	Maestro Visual Testing ou React Native Owl
9	Mobile API Contract Testing ★★	Garantir que app e backend continuam "se entendendo"	Pact + Jest (precisa ativar token TMDB real)
13	POM em Maestro ★★★★★	Separar o seletor de cada elemento de como o teste usa ele	<code>runScript</code> + <code>output</code> do próprio Maestro

Base real: Robot Pattern (Jake Wharton) é padrão vivo aplicável no RNTL que já sabem. Maestro lançou visual testing nativo — trocamos AppliTools/Percy (SaaS pago, cobertura RN fina) por caminho gratuito no ecossistema que já dominam.

 Trilha IA aplicada a testes

#	Tema	Em bom português	Stack
6	Test Generation com LLM ★★	IA escreve teste a partir de descrição em português	Claude API + Maestro (Claude Code é substituto sem custo)
7	AI Agent Exploratory ★★★	IA explora o app sozinha, tipo usuário curioso	Claude Code + Maestro MCP

Trocamos AppAgent/DroidBot (código de pesquisa, setup frágil) por Maestro MCP — servidor oficial lançado fev/2026: Claude Code controla emulador direto, app de 25 telas varre em 90s-3min. AppAgent vira bônus pra quem quiser o desafio de pesquisa extra.

Trilha Performance & Segurança

#	Tema	Em bom português	Stack
3	Performance Mobile Testing ★★★★★ (aprofunda o lab de 02/07)	Mede se o app é rápido de verdade	Flashlight + Reassure
4	Mobile Security Testing ★★★★★ (aprofunda o lab de 02/07)	Caça falha de segurança antes do atacante	OWASP MASVS (obrigatório) + MobSF (bônus, exige Docker)

Trocamos Macrobenchmark (engenharia Android nativa avançada, nunca ensinada) por Flashlight (open source, BAM, "Lighthouse pra mobile") + Reassure (Callstack, regressão de render via Jest).

Caso real de impacto: SDK EngageLab expôs "millions of Android wallets" por misconfig de manifest similar ao achado do lab (Microsoft Security Blog, abr/2026).

 Trilha Qualidade & Pipeline

#	Tema	Em bom português	Stack
8	CI/CD Pipeline Mobile ★★★★★	Esteira automática que testa/builda a cada push	Actions+Fastlane+Firebase
11	Mutation Testing ★★★	Testa se os TESTES realmente pegam erro	Stryker + relatório de mutation score

Flaky test quarantine é prática documentada desde o Google Testing Blog (2016) — ~1.5% flake rate afeta ~16% dos testes. StrykerJS documenta suporte real a Jest/React.

 Trilha Manual, Exploratório & Acessibilidade

#	Tema	Em bom português	Stack
10	Accessibility Testing Mobile ★★	App funciona pra quem usa leitor de tela também	a11y Scanner + Inspector + WCAG
12	Manual → Regressão Automatizada ★★ ★	Acha bug com técnica publicada, não "na sorte"	Heurística recente + teste automatizado

Tema 12 é pra quem curte testar manualmente e não quer o projeto inteiro em código —
Parte A: escolher 1 heurística dos últimos 3 anos (ex.: **TQED** — Roman et al., ICTSS 2023;
ou **Gamified Exploratory Testing** — Coppola et al., IEEE TSE 2023/2024) e aplicar num app conhecido. **Parte B:** bugs achados viram teste automatizado, rodando e mostrando passando.

Onde achar material de apoio

3 pastas prontas, já curadas.

- `exercicios/04-trabalho-final/temas/NN-slug/README.md` — **1 por tema**, execução passo a passo (o que é, como rodar, o que entregar)
- `material-de-apoio/artigos-referencia/` — papers + recursos práticos RN

Pasta 1 — Papers acadêmicos

Curadoria de artigos indicados por ferramenta de busca acadêmica, **verificados manualmente** antes de virar referência da disciplina.

8 papers válidos baixados — inclui METFORD (mutation testing Android, Journal of Systems and Software), Systemic Flakiness (arXiv, aceito no EASE 2025), RAVEN (simpósio SBC).

3 marcados como revista predatória — Sarcouncil, wr-publishing.org, AIRCC. Todos com o mesmo padrão: estatística "redonda demais" (91%, 68%, 85%...) sem journal/instituição rastreável.

Como reconhecer revista predatória (lição pro relatório de vocês)

- **Escopo vago demais** — "Journal of Everything Technology"
- **Editorial board sem credencial verificável**
- **"Rapid publication turnaround"** anunciado como vantagem — contradiz peer review sério
- **Estatística redonda demais no abstract** sem afiliação/venue rastreável
- **Sem indexação** em Scopus/Web of Science/DBLP

Checar **Beall's List** (beallslist.net) quando a fonte parecer boa demais pra ser verdade.

Vale pro relatório do trabalho final: citação acadêmica precisa disso.

Pasta 2 — Recursos práticos (RN-first)

Blog/tutorial/YouTube **específico de React Native**, 1+ por tema — não é Android genérico.

- Doc oficial (Maestro, Stryker, React Native, **blog oficial da OWASP sobre RN**)
- Empresa real em produção (Sentry, Callstack, Wix, Cars24, Very Good Ventures)
- Casos quantificados (ex.: 20+ apps white-label, cold start -40% com New Architecture+Hermes)

Tema 7 (AI Agent Exploratory) não tem material RN-specific — AppAgent/DroidBot leem view tree Android nativa, não componente RN. É característica da ferramenta, não lacuna de pesquisa.

Rubrica (60 pts) — recap

Critério	Pts
Apresentação oral (10min + 5min Q&A)	12
Artefato técnico (repo funcional, executando)	21
Relatório analítico (1pg, ≥3 referências)	12
Domínio conceitual em Q&A	9
Originalidade e profundidade	6

CI não é obrigatório. Basta rodar os testes/flow e mostrar passando (print, log ou vídeo no README). CI real (Actions com badge) soma em "originalidade e profundidade", não é requisito.

Entrega: 15/07/2026. Enunciado completo: `exercicios/04-trabalho-final/enunciado.md`.

Dúvidas?

Teams da turma ou jackson.96@gmail.com

Escolham pela skill que querem desenvolver — não só pelo tema "mais IA".